

AI Genie 專案交接文件

專案名稱：AI Genie — 多租戶聊天機器人平台
GitHub 倉庫：<https://github.com/YwY170/Web-Chatbot-Platform>
撰寫日期：2026-05-22
撰寫人：Wendy YU
授權：GNU AGPL-3.0

一、專案概述

AI Genie 是一套可嵌入任意網站的多租戶 AI 聊天機器人平台，核心特色為：

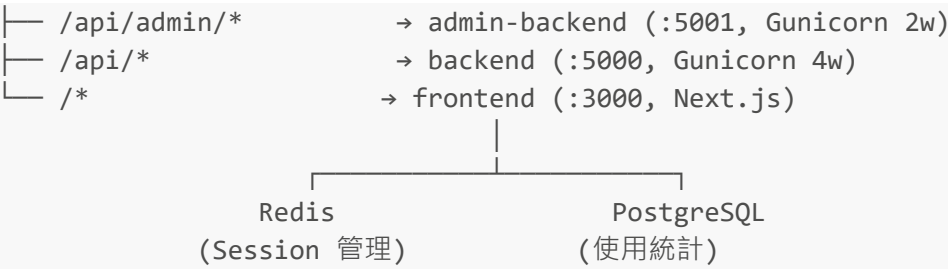
- 每個品牌（租戶）擁有獨立的 API Key、服務設定、提示詞、外觀與翻譯
- 支援 AI 意圖路由，自動判斷使用者意圖並分派至對應服務
- 提供 Web 管理後台，可即時編輯提示詞、快捷按鈕、外觀設定
- 一行 `<script>` 即可嵌入任意網站
- 支援多語系（繁中、英、日、韓、越、印尼、泰）介面自動翻譯

二、技術架構總覽

層級	技術
前端框架	Next.js 16 (App Router) + React 19 + TypeScript 5
前端樣式	Tailwind CSS 4 + Framer Motion
後端框架	Flask 3.0 (Python 3.11+)
AI 服務	Google Gemini 2.5 Flash (google-genai SDK)
Session 管理	Redis 5.0 (TTL 1 小時)
關聯式資料庫	PostgreSQL 16 (數據統計)
容器化	Docker + Docker Compose
反向代理	Nginx Alpine
生產 WSGI	Gunicorn (主 API 4 workers , 管理 API 2 workers)
圖片處理	Pillow

系統請求流程





三、專案目錄結構

chatbot-platform/	
└─ app/	# Next.js App Router 頁面
└─┬─ page.tsx	# 首頁 (重導至管理後台)
└─┬─ layout.tsx	# 根佈局
└─┬─ globals.css	# 全域樣式 (Tailwind)
└─┬─ chat/page.tsx	# 聊天主頁面
└─┬─ admin/	# 管理後台
└─┬─┬─ layout.tsx	# 管理後台 Layout (側邊欄)
└─┬─┬─ page.tsx	# 重定向至品牌列表
└─┬─┬─ login/page.tsx	# 登入頁
└─┬─┬─ stats/page.tsx	# 品牌數據總覽
└─┬─┬─┬─ tenants/	
└─┬─┬─┬─┬─ page.tsx	# 品牌列表 + 嵌入碼
└─┬─┬─┬─┬─┬─ [id]/	
└─┬─┬─┬─┬─┬─┬─ page.tsx	# 品牌編輯 + 翻譯管理
└─┬─┬─┬─┬─┬─┬─┬─ services/	# 服務管理
└─┬─┬─┬─┬─┬─┬─┬─┬─ appearance/	# 外觀設定 (即時預覽)
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─ dashboard/	# 單一品牌數據儀表板
└─┬─ api/	
└─┬─┬─ chat/route.ts	# 聊天 API 代理
└─┬─┬─ chat-widget/route.ts	# 嵌入式腳本產生器
└─ src/	
└─┬─ components/	
└─┬─┬─ chat/	# ChatContainer, MessageList, InputBar 等
└─┬─┬─ ui/	# Button, Card, Carousel
└─┬─ lib/	
└─┬─┬─ api-client.ts	# 前端 → 後端 API 通訊
└─┬─┬─ admin/api-client.ts	# 管理後台 API Client
└─┬─┬─ appearance.ts	# 外觀工具函式
└─┬─┬─ i18n.ts	# 多語系
└─┬─┬─ utils.ts	# 通用工具函式
└─┬─ types/index.ts	# TypeScript 型別定義
└─┬─ contexts/LanguageContext.tsx	# React Context (語言)
└─┬─ locales/translations.json	# 前端翻譯檔
└─ backend/	
└─┬─ app.py	# 主 API 伺服器 (port 5000)
└─┬─ admin_app.py	# 管理後台 API (port 5001)
└─┬─ db.py	# PostgreSQL 資料庫操作
└─┬─ config/	
└─┬─┬─ tenant_manager.py	# 租戶設定管理器

├─ service_factory.py	#	服務工廠 (動態建立 AI 服務)
├─ frappe_templates.py	#	Frappe ERP 模組模板定義
├─ tenants.json	#	租戶設定資料 (本機開發用)
├─ services/		
│ ├─ base_gemini_service.py	#	Gemini AI 基礎類別 + Redis session
│ ├─ chat_service.py	#	通用對話問答服務
│ ├─ query_service.py	#	資料查詢服務 (含 grounding + 快取)
│ ├─ smart_route_service.py	#	路線導航服務 (Google Maps 工具)
│ └─ frappe_query_service.py	#	Frappe ERP 資料查詢服務
├─ middleware/		
│ └─ tenant_auth.py	#	租戶驗證中介層
├─ test/	#	測試檔案
│ ├─ test_tenant_manager.py	#	租戶管理器測試
│ ├─ test_service_factory.py	#	服務工廠測試
│ ├─ test_tenant_auth.py	#	租戶驗證測試
│ ├─ test_admin_api.py	#	管理 API 測試
│ ├─ test_admin_prompts.py	#	提示詞管理測試
│ ├─ test_admin_services.py	#	服務管理測試
│ ├─ test_frappe_service.py	#	Frappe 服務測試
│ └─ test_integration.py	#	整合測試
├─ prompts/{tenant_id}/	#	各租戶提示詞 (Markdown)
├─ translations/{tenant_id}/	#	各租戶多語系翻譯檔 (JSON)
├─ data/	#	Docker volume 掛載目錄
│ ├─ tenants.json	#	生產用租戶設定
│ └─ images/tenants/	#	租戶上傳圖片
├─ Dockerfile	#	主 API 容器 (Gunicorn 4w)
├─ Dockerfile.admin	#	管理 API 容器 (Gunicorn 2w)
├─ .env.example	#	後端環境變數範本
├─ requirements.txt	#	Python 依賴套件
├─ public/	#	靜態資源 (圖示、測試頁面)
├─ docs/		
│ └─ APPEARANCE_GUIDE.md	#	外觀設定指南
├─ docker-compose.yml	#	容器編排 (6 服務)
├─ nginx.conf	#	Nginx 反向代理設定
├─ Dockerfile	#	前端容器 (多階段建置)
├─ next.config.ts	#	Next.js 設定
├─ tsconfig.json	#	TypeScript 設定
├─ eslint.config.mjs	#	ESLint 設定
├─ postcss.config.mjs	#	PostCSS 設定
├─ package.json	#	前端依賴與腳本
├─ .env.example	#	根目錄環境變數範本
└─ LICENSE	#	AGPL-3.0

3.1 功能模組分類

以下按功能將專案中的程式檔案分類，方便快速定位相關程式碼：

AI 服務模組 ([backend/services/](#))

核心 AI 對話與查詢邏輯，所有服務皆繼承 [BaseGeminiService](#)。

```

└─ base_gemini_service.py      # Gemini AI 基礎類別 ( Redis session、grounding、URL
    並行處理 )
└─ chat_service.py            # 通用對話問答服務
└─ query_service.py           # 資料查詢服務 ( 含 Google Search grounding + 快取 )
└─ smart_route_service.py      # 路線導航服務 ( Google Maps Function Calling )
└─ frappe_query_service.py     # Frappe ERP 資料查詢服務 ( REST API + AI 模糊配對 )

```

⚙️ 租戶管理與設定模組 ([backend/config/](#))

多租戶架構的核心設定層，負責租戶 CRUD、服務動態建立。

```

└─ tenant_manager.py          # 租戶設定管理器 ( 讀寫 tenants.json )
└─ service_factory.py         # 服務工廠 ( 依 class 名稱動態建立服務實例 )
└─ frappe_templates.py        # Frappe ERP 模組模板定義
└─ tenants.json               # 租戶設定資料 ( 所有品牌的服務、外觀、Quick Actions )

```

🔒 中介層模組 ([backend/middleware/](#))

請求驗證與租戶識別。

```

└─ tenant_auth.py            # 租戶驗證裝飾器 ( 驗證 X-Tenant-ID / X-Admin-Key )

```

🌐 API 伺服器模組 ([backend/](#))

Flask 應用程式入口，分為聊天 API 與管理 API。

```

└─ app.py                    # 聊天主 API ( port 5000 ) – 意圖判斷、對話、查詢
└─ admin_app.py              # 管理後台 API ( port 5001 ) – 租戶/服務/外觀 CRUD
└─ db.py                     # PostgreSQL 資料層 ( 訊息記錄、Session 統計 )

```

💬 前端聊天 UI 模組 ([src/components/chat/](#))

聊天介面的 React 元件，負責訊息顯示、輸入、快捷操作。

```

└─ ChatContainer.tsx         # 主容器 ( 狀態管理、API 呼叫、意圖路由 )
└─ ChatHeader.tsx           # 標題列 ( 品牌名稱、語言切換 )
└─ MessageList.tsx          # 訊息列表 ( 自動捲動 )
└─ Message.tsx              # 單則訊息 ( Markdown 渲染、參考連結 )
└─ InputBar.tsx             # 輸入框 ( 送出、禁用狀態 )
└─ QuickActions.tsx         # 快速操作按鈕列
└─ LoadingMessage.tsx       # 載入中動畫

```

👤 前端外觀與 UI 元件 (`src/components/ui/`)

通用 UI 元件庫。

```
├─ Button.tsx      # 按鈕元件
├─ Card.tsx        # 卡片元件
└─ Carousel.tsx    # 輪播元件
```

🔗 前端工具與 API 通訊 (`src/lib/`)

前端與後端的通訊層、多語系、外觀工具。

```
├─ api-client.ts    # Chat API Client ( 聊天、意圖、語言偵測 )
├─ admin/api-client.ts # Admin API Client ( 租戶/服務/外觀/翻譯 CRUD )
├─ appearance.ts    # 外觀型別定義與工具函式
├─ i18n.ts          # 多語系工具 ( 翻譯載入、語言切換 )
└─ utils.ts         # 通用工具函式
```

📄 管理後台頁面 (`app/admin/`)

Next.js App Router 管理後台頁面。

```
├─ layout.tsx      # 管理後台 Layout ( 側邊欄導航 )
├─ page.tsx        # 首頁 ( 重定向至品牌列表 )
├─ login/page.tsx  # 登入頁 ( 驗證 Admin API Key )
├─ stats/page.tsx  # 全域數據統覽
└─ tenants/
  ├─ page.tsx      # 品牌列表 + 嵌入碼產生
  └─ [id]/
    ├─ page.tsx    # 品牌基本設定 + 翻譯管理
    ├─ services/   # 服務設定 CRUD 頁面
    ├─ appearance/ # 外觀設定 ( 即時預覽 )
    └─ dashboard/  # 單一品牌數據儀表板
```

🌐 多語系與翻譯模組

前端 UI 翻譯與後端 AI 生成翻譯。

```
├─ src/locales/translations.json # 前端 UI 靜態翻譯檔
├─ src/contexts/LanguageContext.tsx # React Context ( 語言狀態管理 )
├─ backend/translations/{tenant_id}/ # 各租戶 AI 生成的多語系翻譯 ( JSON )
└─ src/lib/i18n.ts               # 多語系工具函式
```

 提示詞管理 ([backend/prompts/](#))

各租戶各服務的 AI 提示詞，Markdown 格式，可透過管理後台線上編輯。

```
└─ {tenant_id}/
  │ general.md      # 通用對話提示詞
  │ query.md        # 資料查詢提示詞
  │ route.md        # 路線導航提示詞
  │ ...             # 其他自訂服務提示詞
```

 容器化與部署 (根目錄)

Docker 容器編排與反向代理設定。

```
└─ docker-compose.yml      # 容器編排 ( 6 服務 : redis, postgres, backend, admin-backend, frontend, nginx )
└─ nginx.conf              # Nginx 反向代理 ( 路由分發 + 靜態圖片 serve )
└─ Dockerfile              # 前端容器 ( 多階段建置 : deps → build → runner )
└─ backend/Dockerfile      # 主 API 容器 ( Gunicorn 4 workers )
└─ backend/Dockerfile.admin # 管理 API 容器 ( Gunicorn 2 workers )
```

 嵌入式 Widget ([app/api/](#))

一行 script 嵌入聊天機器人的產生器。

```
└─ chat-widget/route.ts    # 嵌入式 JS 產生器 ( 浮動按鈕 + iframe )
└─ chat/route.ts           # 聊天 API 代理 ( placeholder )
```

四、核心模組說明

4.1 後端核心服務 ([backend/services/](#))

類別	檔案	說明
BaseGeminiService	base_gemini_service.py	所有 AI 服務的基礎類別；提供 Redis session 管理、grounding URL 並行處理、溫度/關鍵字配置、Markdown 提示詞載入、多語言回覆
ChatService	chat_service.py	通用對話問答，繼承 BaseGeminiService
QueryService	query_service.py	資料查詢，支援 Gemini grounding 搜尋 + 快取
SmartRouteService	smart_route_service.py	路線導航規劃，整合 Google Maps Function Calling

類別	檔案	說明
FrappeQueryService	(services/)	串接 Frappe ERP REST API，支援 AI 模糊配對查詢

4.2 租戶管理 (backend/config/)

- **tenant_manager.py**：讀寫 **tenants.json**，提供租戶 CRUD、服務設定、外觀設定、Quick Actions 管理。Docker 環境下設定檔路徑由環境變數 **TENANTS_CONFIG_PATH** 指定 (預設掛載至 **/app/data/tenants.json**)。
- **service_factory.py**：動態建立 AI 服務實例，維護 **SERVICE_CLASSES** 字典，新增自訂服務類別需在此處註冊。

4.3 API 伺服器

app.py (port 5000) — 聊天主 API

方法	端點	說明
GET	/api/health	健康檢查
POST	/api/detect-language	語言偵測
POST	/api/chat/intent	AI 意圖判斷
POST	/api/chat	聊天 (mode 指定服務)
GET	/api/tenant/config	取得租戶前端設定 (支援 ?lang=)
GET	/api/query/data	查詢資料 (支援 ?service=)

所有聊天 API 需傳遞 **X-Tenant-ID** Header。

admin_app.py (port 5001) — 管理後台 API

方法	端點	說明
GET/POST	/api/admin/tenants	列出 / 建立租戶
GET/PUT/DELETE	/api/admin/tenants/{id}	取得 / 更新 / 刪除租戶
GET/POST/PUT/DELETE	/api/admin/tenants/{id}/services	服務 CRUD
GET/PUT	/api/admin/tenants/{id}/quick-actions	快捷按鈕管理
GET/PUT	/api/admin/tenants/{id}/appearance	外觀設定
POST	/api/admin/tenants/{id}/appearance/upload-icon	上傳聊天圖示
GET/POST	/api/admin/tenants/{id}/translations	翻譯狀態 / 生成翻譯

所有管理 API 需傳遞 **X-Admin-Key** Header (值為 **backend/.env** 中的 **ADMIN_API_KEY**)。

4.4 資料庫 (backend/db.py)

PostgreSQL 16 用於統計用途，連線字串由環境變數 `DATABASE_URL` 提供。Docker 環境下透過 `docker-compose.yml` 注入，格式：

```
postgresql://aigenie:${POSTGRES_PASSWORD}@postgres:5432/aigenie
```

五、環境變數說明

5.1 根目錄 `.env` (前端 + Nginx)

```
NGINX_PORT=80
NEXT_PUBLIC_API_URL=https://your-domain.com      # 聊天 API URL ( 生產環境 )
NEXT_PUBLIC_ADMIN_API_URL=https://your-domain.com # 管理 API URL ( 生產環境 )
NEXT_PUBLIC_FRONTEND_URL=https://your-domain.com  # 前端 URL ( 生產環境 )
```

本機開發時：`NEXT_PUBLIC_API_URL=http://localhost:5000`，依此類推。

5.2 `backend/.env` (後端服務)

```
# AI 服務
GEMINI_API_KEY=your_gemini_api_key_here

# 管理後台密碼 ( 即 X-Admin-Key )
ADMIN_API_KEY=your_admin_secret_key

# Redis
REDIS_HOST=localhost      # Docker 環境下由 docker-compose 覆蓋為 "redis"
REDIS_PORT=6379
REDIS_DB=0
REDIS_PASSWORD=

# PostgreSQL
POSTGRES_PASSWORD=your_strong_password
DATABASE_URL=postgresql://aigenie:your_strong_password@localhost:5432/aigenie

# Frappe ERP ( 選填，使用 FrappeQueryService 時才需設定 )
FRAPPE_URL=
FRAPPE_API_KEY=
FRAPPE_API_SECRET=
```

重要：各租戶的 Gemini API Key 由管理後台建立租戶時自動寫入 `tenants.json`，格式為 `TENANT_{TENANT_ID}_GEMINI_API_KEY`。

六、部署說明

6.1 本機開發部署

前置需求：Node.js 20+、Python 3.11+、Redis、PostgreSQL (可選)

```
# 步驟 1：安裝前端依賴
npm install

# 步驟 2：安裝後端依賴
cd backend
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt

# 步驟 3：設定環境變數
cp .env.example .env              # 根目錄
cp backend/.env.example backend/.env
# 編輯兩個 .env 填入實際值

# 步驟 4：啟動 Redis
redis-server
# 或 Docker 方式：
docker run -d --name redis -p 6379:6379 redis:latest

# 步驟 5：啟動後端 ( 開兩個終端 )
# 終端 1
cd backend && source venv/bin/activate && python app.py

# 終端 2
cd backend && source venv/bin/activate && python admin_app.py

# 步驟 6：啟動前端 ( 第三個終端 )
npm run dev
```

各服務存取位址

頁面	URL
聊天頁面	http://localhost:3000/chat?tenant_id=demo
管理後台	http://localhost:3000/admin
管理後台登入	http://localhost:3000/admin/login
後端健康檢查	http://localhost:5000/api/health
管理 API 健康檢查	http://localhost:5001/api/admin/health

6.2 Docker 生產部署

前置需求：Docker Engine 24+、Docker Compose v2+

```
# 步驟 1：設定環境變數
cp .env.example .env
cp backend/.env.example backend/.env
# 編輯 .env 和 backend/.env，填入生產環境值

# 步驟 2：準備資料目錄 (tenants.json volume 掛載)
mkdir -p backend/data backend/data/images/tenants
cp backend/config/tenants.json backend/data/tenants.json

# 步驟 3：建置並啟動所有服務
docker-compose up -d --build
```

Docker Compose 服務一覽

服務名稱	容器名稱	Port	說明
redis	ai-genie-redis	內部	Redis，啟用 AOF 持久化
postgres	ai-genie-postgres	127.0.0.1:5432	PostgreSQL 16，資料統計
backend	ai-genie-backend	內部 5000	主聊天 API，Gunicorn 4 workers
admin-backend	ai-genie-admin-backend	內部 5001	管理 API，Gunicorn 2 workers
frontend	ai-genie-frontend	內部 3000	Next.js 前端
nginx	ai-genie-nginx	\${NGINX_PORT}:80	反向代理 + 靜態圖片

常用維運指令

```
docker-compose up -d          # 啟動所有服務
docker-compose down           # 停止所有服務
docker-compose up -d --build  # 重新建置並啟動 (程式碼更新後執行)
docker-compose logs -f        # 查看即時日誌 (全部服務)
docker-compose logs -f backend # 查看特定服務日誌
docker-compose restart backend # 重啟特定服務
```

Volume 掛載說明

Volume / 掛載路徑	說明
redis-data	Redis AOF 持久化資料
pg_data	PostgreSQL 資料
./backend/data:/app/data	tenants.json + 其他設定
./backend/prompts:/app/prompts	各租戶提示詞 (Markdown)
./backend/translations:/app/translations	各租戶翻譯檔 (JSON)
./backend/data/images/tenants:/app/public/images/tenants	租戶上傳圖片

七、嵌入聊天機器人

在目標網站的 HTML 加入以下一行即可啟用聊天機器人：

```
<!-- 生產環境 -->
<script src="https://your-domain.com/api/chat-widget?tenant_id=YOUR_TENANT_ID">
</script>

<!-- 本機開發測試 -->
<script src="http://localhost:3000/api/chat-widget?tenant_id=demo"></script>
```

也可使用 `public/test.html` 進行本機嵌入效果測試。

八、新增自訂 AI 服務

1. 在 `backend/services/` 建立新的服務類別，繼承 `BaseGeminiService`
2. 在 `backend/config/service_factory.py` 的 `SERVICE_CLASSES` 字典中註冊新類別
3. 在 `backend/prompts/{tenant_id}/` 底下建立對應的 Markdown 提示詞檔案
4. 透過管理後台 → 品牌管理 → 新增服務，選擇對應的服務類別

BaseGeminiService 提供的功能

- Redis 使用者 Session 管理 (TTL 1 小時)
- 並行 URL 處理，取得 grounding 參考資料
- 可設定的 `temperature`、`grounding`、`search_keyword`
- 從 Markdown 檔案動態載入 system prompt
- 多語言回覆支援

九、測試

```
# 執行所有後端測試
cd backend
python -m pytest test/ -v

# 個別測試
python test/test_tenant_manager.py
python test/test_service_factory.py
python test/test_tenant_auth.py
python test/test_admin_api.py
python test/test_admin_prompts.py
python test/test_admin_services.py
python test/test_integration.py
```

十、常用開發指令速查

```
# 前端開發
npm run dev          # 啟動 Next.js 開發伺服器 (port 3000)
npm run build         # 正式環境建置
npm start             # 啟動正式環境伺服器
npm run lint          # 執行 ESLint

# 後端測試
cd backend && python -m pytest test/ -v

# Docker 操作
docker-compose up -d      # 啟動
docker-compose down       # 停止
docker-compose up -d --build # 重建後啟動
docker-compose logs -f    # 即時日誌
```

十一、注意事項與常見問題

1. **tenants.json 路徑差異**：本機開發時讀取 `backend/config/tenants.json`，Docker 生產環境讀取由環境變數 `TENANTS_CONFIG_PATH` 指定的路徑（預設 `/app/data/tenants.json`），初次部署前務必執行 `cp backend/config/tenants.json backend/data/tenants.json`。
2. **Redis 連線**：本機開發預設連 `localhost:6379`；Docker 環境下 `REDIS_HOST` 由 `docker-compose` 自動覆蓋為 `redis`（服務名稱），無需手動修改 `backend/.env`。
3. **NEXT_PUBLIC_ 環境變數**：Next.js 前端的 `NEXT_PUBLIC_*` 變數在**建置時**即被編譯進去，Docker 部署時需在 `docker-compose.yml` 的 `args` 區段傳入，若部署後更換 Domain 需重新 `--build`。
4. **管理後台登入**：密碼即為 `backend/.env` 的 `ADMIN_API_KEY` 值，請設定足夠複雜的字串。
5. **圖片靜態服務**：租戶上傳的圖示由 Nginx 直接 serve，路徑為 `/images/tenants/*`，對應宿主機目錄 `./backend/data/images/tenants/`。
6. **Frappe ERP 整合**：`FrappeQueryService` 為選用服務，僅在設定 `FRAPPE_URL`、`FRAPPE_API_KEY`、`FRAPPE_API_SECRET` 後才能正常運作。
7. **翻譯過期偵測**：管理後台翻譯 API 提供過期偵測功能，若原始文案更新後翻譯未同步，系統會標記為過期狀態，可於後台手動觸發重新生成。

十二、聯絡資訊

項目	資訊
原始開發者	Wendy YU
GitHub	https://github.com/YwY170
倉庫	https://github.com/YwY170/Web-Chatbot-Platform
授權	GNU AGPL-3.0 (含第 7 條額外條款)

十三、貢獻與溝通準則

本專案歡迎任何形式的技術交流與改進建議。若您發現程式架構有需要改進之處，歡迎透過 [Issue](#) 或 [Pull Request](#) 提出公開、具體、建設性的反饋。

本專案明確拒絕任何以私下或公開方式貶低、嘲諷原創作者開發瑕疵的行為。

每一段程式碼都是作者投入心力的成果，技術本就是在迭代中進步的。請以尊重與善意的方式進行交流，共同維護良好的開源協作環境。

本專案支持性別友善，明確拒絕任何形式的性別攻擊與歧視。

無論性別認同或性別表達為何，每位貢獻者與使用者都應受到平等尊重。